# FastAPI and ML Integrated AI-Agent

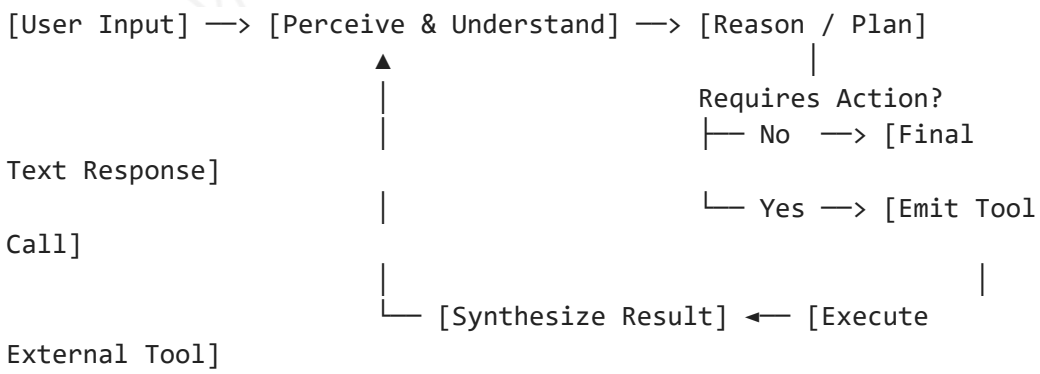
An **AI Agent** is an autonomous system that uses a Large Language Model (LLM) as its reasoning engine to perceive user intent, plan actions, interact with external environments or tools, and achieve a specific goal. Unlike a traditional chatbot that only generates text from internal pre-trained memory, an AI agent can execute code, query databases, invoke APIs, and make decisions based on dynamic feedback.

---

## 1. Core AI Agent Architecture & Concepts

### AI Agent Lifecycle

The lifecycle of an AI agent follows a continuous loop:




1. **Perception**: The agent reads user input along with existing conversational history and system instructions.
2. **Reasoning & Planning**: The LLM evaluates whether it can answer directly or if it requires missing information or external tools to fulfill the user's intent.
3. **Action (Tool Invocation)**: If an external action is required, the LLM emits a structured instruction specifying which function to call and with what exact arguments.
4. **Observation**: The host application executes the function and feeds the resulting data back into the LLM context.
5. **Synthesis**: The LLM analyzes the tool output against the initial prompt to produce a natural-language response or plan the next action.

## Usage of the OpenAI Client

The `openai` Python SDK serves as an industry-standard interface for chat completions, streaming, and tool execution. Many non-OpenAI model providers (such as Groq, Mistral, Together AI, or local runners like Ollama and vLLM) implement OpenAI-compatible REST APIs.

In your application, configuring:

```
client = OpenAI(
    api_key=os.environ.get("GROQ_API_KEY"),
    base_url="https://api.groq.com/openai/v1"
)
```

swaps the underlying inference backend from OpenAI's servers to Groq's high-speed LPU infrastructure while preserving the exact same client methods (`client.chat.completions.create(...)`).

## Tool Function vs. Tool Schema

In tool-augmented LLM architectures, there is a clear distinction between the **tool specification** and the **tool execution**:

- **Tool Schema (JSON Schema)**: Tells the LLM *what* functions exist, what they do, and what parameters they require. The LLM itself **never executes Python code directly**; it reads this schema and outputs structured JSON conforming to your specification.
- **Tool Function (Python Implementation)**: The actual code living in your application environment that performs the network request, queries a database, or triggers an ML inference job when requested by the model.

## Session State Messages & The Message Schema

LLM APIs are stateless: they do not remember previous interactions. To maintain conversation context, the entire transcript is sent in every request as a list of message objects:

- `{"role": "system", "content": "..."}`: High-level instructions governing behavior, guardrails, and role personas.
- `{"role": "user", "content": "..."}`: Queries submitted by the end user.
- `{"role": "assistant", "content": "...", "tool_calls": [...]}`: Output generated by the LLM. If requesting a tool call, `tool_calls` lists the function name, unique invocation ID, and serialized JSON arguments.
- `{"role": "tool", "tool_call_id": "...", "name": "...", "content": "..."}`: The raw return value of the executed Python tool, matched back to the corresponding call via `tool_call_id`.

## Two-Step Tool Calling Cycle

When an LLM decides to use an external tool, the transaction requires two distinct API calls:

```
[Turn 1: Request]   User Prompt + Conversation History + Available
Tools
[Turn 1: Response]  LLM returns tool_calls (e.g.,
get_house_price_estimate(sqft=2000...))
                              |
                    (Python executes get_house_price_estimate() via
FastAPI)
                              |
[Turn 2: Request]   User Prompt + Assistant tool_call + Tool
execution result
[Turn 2: Response]  LLM synthesizes final advisory response in
natural language
```

1. **First Call (Trigger)**: You pass user messages and tool schemas. The model assesses the prompt, notices missing information (and asks follow-up questions) or determines that all arguments exist and emits a `tool_calls` object.
2. **Second Call (Synthesis)**: Your script executes the function locally, captures the output, appends a `role: tool` message to the message array, and invokes the API again. The LLM processes the tool output in context and drafts a human-readable synthesis.

---

# 2. UI & Serving Architecture: Streamlit & FastAPI

## What is Streamlit and Its Purpose?

Streamlit is an open-source Python framework designed to build interactive data and AI web frontends quickly without requiring HTML, CSS, JavaScript, or dedicated frontend routing.

### Core Streamlit Concepts

- **Execution Model (Full Rerun)**: Streamlit runs the script from top to bottom every time a user interacts with a widget (clicks a button, types in an input box, etc.).
- **`st.session_state`**: A persistent key-value store across script reruns for a given browser tab. Without `st.session_state.messages`, the conversation history would wipe out entirely on every new keystroke or button press.
- **`st.chat_message` & `st.chat_input`**: Built-in chat interface elements that render message containers styled by role (`user` or `assistant`) and capture text inputs cleanly.

## Why FastAPI is Integrated Here

Instead of placing the Scikit-Learn model directly inside the Streamlit script, the model is decoupled into a dedicated FastAPI service. This architecture provides critical advantages:

- **Separation of Concerns**: Machine learning inference and model weight management remain completely isolated from UI rendering and agent conversation flow.
- **Independent Scalability**: In production, the ML server can scale horizontally on compute- or GPU-optimized nodes, while the frontend runs on lighter web servers.
- **Framework Agnosticism**: Any client (Streamlit app, mobile app, internal batch pipeline, or another agent) can consume predictions via standard HTTP JSON contracts.
- **Strict Validation**: FastAPI leverages Pydantic to ensure all incoming data conforms to required types before running Scikit-Learn inference.

---

# 3. Step-by-Step Code Walkthrough

## 1. `house_price_model.py` (Model Training & Export)

This file generates synthetic data, trains a regression model, and persists the trained artifact to disk.

```
# 1. Synthetic Data Generation
np.random.seed(42)
n_samples = 1000

sqft = np.random.randint(800, 5000, n_samples)
bedrooms = np.random.randint(1, 6, n_samples)
bathrooms = np.random.randint(1, 4, n_samples)
age = np.random.randint(0, 50, n_samples)
location = np.random.randint(1, 11, n_samples)
```

Generates 1,000 realistic, randomized data points across five property features.

```
# 2. Synthetic Ground-Truth Formula + Market Noise
base_price = 100000
price = (base_price + (sqft * 160) + (bedrooms * 12000) +
         (bathrooms * 8000) - (age * 1500) + (location * 25000))
price = price + np.random.normal(0, 15000, n_samples)
```

Establishes a logical mathematical dependency: square footage, room counts, and location raise value, while house age reduces value. Normal distribution noise models real-world market variance.

```
# 3. Model Training & Serialization
X = pd.DataFrame({'sqft': sqft, 'bedrooms': bedrooms, 'bathrooms':
bathrooms, 'age_of_house': age, 'location_score': location})
y = price

model = RandomForestRegressor(n_estimators=100, random_state=42)
model.fit(X, y)
joblib.dump(model, 'real_estate_model.joblib')
```

- Fits an ensemble of 100 decision trees ( `RandomForestRegressor` ) to map nonlinear feature relationships against property valuation.

- `joblib.dump(...)` serializes the trained model into a binary file (`real_estate_model.joblib`), allowing external processes to load the weights without re-training.

---

## 2. `fast_api.py` (Inference Service)

This file loads the serialized model and serves it via an HTTP POST endpoint.

```
app = FastAPI(title="Vi4 Academy ML Predictor")
model = joblib.load("real_estate_model.joblib")
```

- Instantiates the FastAPI application.
- Loads the trained Scikit-Learn model into server memory at boot time so predictions can be computed immediately.

```
class PropertyFeatures(BaseModel):
    sqft: int
    bedrooms: int
    bathrooms: int
    age_of_house: int
    location_score: int
```

Defines a Pydantic schema enforcing that every incoming request payload contains exactly these five numeric fields with valid integer data types.

```
@app.post("/predict-price")
def predict_price(features: PropertyFeatures):
    input_data = np.array([[
        features.sqft,
        features.bedrooms,
        features.bathrooms,
        features.age_of_house,
        features.location_score
    ]])
    prediction = model.predict(input_data)[0]
    return {"estimated_price": int(prediction)}
```

- **Endpoint routing**: Binds `predict_price` to the `POST /predict-price` route.
- **Feature restructuring**: Converts the validated Pydantic model into a 2D NumPy array of shape `(1, 5)` as expected by Scikit-Learn.
- **Inference**: Computes the predicted price and returns an HTTP JSON response containing the integer estimate.

---

## 3. `AI_Agent.py` (Streamlit Frontend & LLM Orchestration)

This file manages the UI, conversation memory, tool definitions, and LLM reasoning loop.

### Client Configuration

```
client = OpenAI(
    api_key=os.environ.get("GROQ_API_KEY"),
    base_url="https://api.groq.com/openai/v1"
)
```

Points the standard OpenAI SDK to Groq's high-speed inference engine using the model `qwen/qwen3.8-27b`.

### Python Tool Execution Handler

```
def get_house_price_estimate(sqft, bedrooms, bathrooms, age_of_house,
location_score):
    url = "http://127.0.0.1:8000/predict-price"
    payload = {
        "sqft": sqft, "bedrooms": bedrooms, "bathrooms": bathrooms,
        "age_of_house": age_of_house, "location_score": location_score
    }
    response = requests.post(url, json=payload)
    return response.json()
```

Packages arguments extracted by the LLM into a JSON payload, performs an HTTP POST request to the local FastAPI server, and returns the parsed JSON response dictionary (`{"estimated_price": ...}`).

### Tool Schema Definition

```
tools = [
    {
        "type": "function",
        "function": {
            "name": "get_house_price_estimate",
            "description": "Estimate a property price based on house
features. Call this when a user requests a valuation.",
            "parameters": {
                "type": "object",
                "properties": {
                    "sqft": {"type": "integer", "description": "Total
interior area in square feet"},
                    "bedrooms": {"type": "integer", "description": "Total
number of bedrooms"},
                    "bathrooms": {"type": "integer", "description": "Total
number of bathrooms"},
                    "age_of_house": {"type": "integer", "description":
"Age of the house in years"},
                    "location_score": {"type": "integer", "description":
"Location rating scale from 1 (poor) to 10 (prime)"}
                },
                "required": ["sqft", "bedrooms", "bathrooms",
"age_of_house", "location_score"]
            }
        }
    }
]
```

Informs the LLM about the tool. Because all five parameters are marked `required`, the system prompt guides the LLM to ask follow-up questions until every parameter has been supplied by the user.

### Chat State & System Instructions

```
if "messages" not in st.session_state:
    st.session_state.messages = [
        {
            "role": "system",
            "content": (
                "You are an expert Real Estate & Financial Advisor AI. "
                "When users ask for a price estimate, ensure you have all
required details: "
                "sqft, bedrooms, bathrooms, age_of_house, and
location_score (1-10). "
                "If any detail is missing, ask follow-up questions
politely. "
                "Once all parameters are gathered, execute the
get_house_price_estimate tool "
                "and provide strategic financial advice based on the
calculated valuation."
            )
        }
    ]
```

Initializes the chat history on first load. The system prompt specifies:

1. Act as a financial and real estate advisor.
2. Require all 5 attributes before attempting a valuation.
3. Formulate strategic financial advice once the valuation data returns.

### The Interaction Loop

```
if prompt := st.chat_input("Ask about home valuations or financial
advice..."):
    st.session_state.messages.append({"role": "user", "content": prompt})
    ...
    response = client.chat.completions.create(
        model="qwen/qwen3.8-27b",
        messages=st.session_state.messages,
        tools=tools,
        tool_choice="auto"
    )
    response_msg = response.choices[0].message
```

When a user submits a question:

1. The prompt is added to `st.session_state.messages`.
2. The agent sends the full conversation history alongside `tools` to the LLM.
3. If information is missing (e.g., user only said "I have a 3 bedroom home"), `response_msg.tool_calls` is `None`. The LLM responds in natural language asking

for the missing variables (bathrooms, sqft, age, location).

4. If all parameters are present, `response_msg.tool_calls` contains a call to `get_house_price_estimate` with the extracted arguments.

### Executing Tools & Synthesizing Results

```
    if response_msg.tool_calls:
        for tool_call in response_msg.tool_calls:
            if tool_call.function.name == "get_house_price_estimate":
                args = json.loads(tool_call.function.arguments)
                result = get_house_price_estimate(...)

                # Append the tool's result to conversation history
                st.session_state.messages.append({
                    "role": "tool",
                    "tool_call_id": tool_call.id,
                    "name": "get_house_price_estimate",
                    "content": json.dumps(result)
                })

        # Second API call: Synthesizes final advice using the tool's
output
        final_response = client.chat.completions.create(
            model="qwen/qwen3.8-27b",
            messages=st.session_state.messages
        )
        final_content = final_response.choices[0].message.content
        st.session_state.messages.append({"role": "assistant", "content":
final_content})
        with st.chat_message("assistant"):
            st.markdown(final_content)
```

1. Parses `tool_call.function.arguments` from a JSON string into a Python dictionary.
2. Calls `get_house_price_estimate(...)`, which posts the data to FastAPI.
3. Appends the valuation result back to `st.session_state.messages` under the `tool` role.
4. Executes the **second API call** to the LLM. The LLM now has both the user's initial inquiry and the concrete prediction result ( `{"estimated_price": 420000}` ).
5. The LLM translates the raw numerical output into a complete advisory response (such as market positioning, mortgage estimates, or purchase viability) and displays it in the Streamlit UI.

---